quectel-Network&GNSS&File

版本：1.0

日期：2026-02-26

状态：受控/临时文件

文件保密级别：（请勾选 ■ ）		
绝密 □	保密 □	公开 ■

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2026-02-26	1	首次发行	Wells Li

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼　　邮编：200233
电话：+86 21 5108 6236　　邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：http://www.quectel.com/cn/support/sales.htm。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
http://www.quectel.com/cn/support/technical.htm 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。
Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

目录

1 模块概述 7

2 Network 模块 7

2.1. Network 7

2.1.1. 接口概述 7

2.1.2. 函数原型 7

2.1.3. 参数描述 7

2.1.4. 返回值描述 7

2.1.5. 示例 7

2.2. init 8

2.2.1. 接口概述 8

2.2.2. 函数原型 8

2.2.3. 参数描述 8

2.2.4. 返回值描述 8

2.2.5. 示例 8

2.3. deinit 8

2.3.1. 接口概述 8

2.3.2. 函数原型 8

2.3.3. 参数描述 8

2.3.4. 返回值描述 8

2.3.5. 示例 9

2.4. query_usim 9

2.4.1. 接口概述 9

2.4.2. 函数原型 9

2.4.3. 参数描述 9

2.4.4. 返回值描述 9

2.4.5. 示例 9

2.5. attach 9

2.5.1. 接口概述 9

2.5.2. 函数原型 9

2.5.3. 参数描述 9

2.5.4. 返回值描述 9

2.5.5. 示例 10

2.6. is_connected 10

2.6.1. 接口概述 10

2.6.2. 函数原型 10

2.6.3. 参数描述 10

2.6.4. 返回值描述 10

2.6.5. 示例 10

2.7. set_apn 10

2.7.1. 接口概述 10

2.7.2. 函数原型 10

2.7.3. 参数描述 10
2.7.4. 返回值描述 11
2.7.5. 示例 11
2.8. set_scanmode 11
2.8.1. 接口概述 11
2.8.2. 函数原型 11
2.8.3. 参数描述 11
2.8.4. 返回值描述 11
2.8.5. 示例 11
2.9. set_ iotopmode 12
2.9.1. 接口概述 12
2.9.2. 函数原型 12
2.9.3. 参数描述 12
2.9.4. 返回值描述 12
2.9.5. 示例 12

3 GNSS 模块 12
3.1. GNSS 12
3.1.1. 接口概述 12
3.1.2. 函数原型 12
3.1.3. 参数描述 13
3.1.4. 返回值描述 13
3.2. start 13
3.2.1. 接口概述 13
3.2.2. 函数原型 13
3.2.3. 参数描述 13
3.2.4. 返回值描述 13
3.2.5. 示例 13
3.3. stop 13
3.3.1. 接口概述 13
3.3.2. 函数原型 13
3.3.3. 参数描述 14
3.3.4. 返回值描述 14
3.4. get_location 14
3.4.1. 接口概述 14
3.4.2. 函数原型 14
3.4.3. 参数描述 14
3.4.4. 返回值描述 14
3.4.5. 示例 15

4 File 模块 15
4.1. File.open 15
4.1.1. 接口概述 15
4.1.2. 函数原型 16
4.1.3. 参数描述 16

4.1.4. 返回值描述 16
4.1.5. 示例 16
4.2. close 16
4.2.1. 接口概述 16
4.2.2. 函数原型 16
4.2.3. 参数描述 16
4.2.4. 返回值描述 17
4.2.5. 示例 17
4.3. read 17
4.3.1. 接口概述 17
4.3.2. 函数原型 17
4.3.3. 参数描述 17
4.3.4. 返回值描述 17
4.3.5. 示例 17
4.4. write 17
4.4.1. 接口概述 17
4.4.2. 函数原型 18
4.4.3. 参数描述 18
4.4.4. 返回值描述 18
4.4.5. 示例 18
4.5. seek 18
4.5.1. 接口概述 18
4.5.2. 函数原型 18
4.5.3. 参数描述 19
4.5.4. 返回值描述 19
4.5.5. 示例 19
4.6. tell 19
4.6.1. 接口概述 19
4.6.2. 函数原型 19
4.6.3. 参数描述 19
4.6.4. 返回值描述 19
4.6.5. 示例 19
4.7. File.listdir 20
4.7.1. 接口概述 20
4.7.2. 函数原型 20
4.7.3. 参数描述 20
4.7.4. 返回值描述 20
4.7.5. 示例 20
4.8. 类 FileInfo 20
4.8.1. 接口概述 20
4.8.2. 函数原型 21
4.8.3. 参数描述 21
4.8.4. 返回值描述 21
4.8.5. 示例 21

4.9. File.statvfs 21
4.9.1. 接口概述 21
4.9.2. 函数原型 21
4.9.3. 参数描述 21
4.9.4. 返回值描述 21
4.9.5. 示例 21
4.10. File.remove 22
4.10.1. 接口概述 22
4.10.2. 函数原型 22
4.10.3. 参数描述 22
4.10.4. 返回值描述 22
4.10.5. 示例 22
4.11. File.mkdir 22
4.11.1. 接口概述 22
4.11.2. 函数原型 22
4.11.3. 参数描述 22
4.11.4. 返回值描述 23
4.11.5. 示例 23
4.12. File.rmdir 23
4.12.1. 接口概述 23
4.12.2. 函数原型 23
4.12.3. 参数描述 23
4.12.4. 返回值描述 23
4.12.5. 示例 23

5 LOG 模块 24
5.1. Log 24
5.1.1. 接口概述 24
5.1.2. 函数原型 24
5.1.3. 参数描述 24
5.1.4. 返回值描述 24
5.2. set_level 24
5.2.1. 接口概述 24
5.2.2. 函数原型 24
5.2.3. 参数描述 24
5.2.4. 返回值描述 25
5.2.5. 示例 25
5.3. get_level 25
5.3.1. 接口概述 25
5.3.2. 函数原型 25
5.3.3. 参数描述 25
5.3.4. 返回值描述 25

1 模块概述

quectel 模块提供了对移远通信模块特定硬件功能的访问接口。当前版本包含两个子模块：Network 用于蜂窝网络连接管理，GNSS 用于全球导航卫星系统定位，FILE 模块提供了对移远通信模块文件系统的访问接口。通过该模块，您可以执行文件读写、目录列表、文件删除、空间查询以及文件上传等操作。 LOG 模块用于设置和查询系统日志输出级别。

2 Network 模块

2.1. Network

2.1.1. 接口概述

创建一个新的 Network 对象实例。

2.1.2. 函数原型

network = quectel.Network()

2.1.3. 参数描述

无。

2.1.4. 返回值描述

返回一个新的 Network 对象。

2.1.5. 示例

```
import quectel

network = quectel.Network()
```

2.2. init

2.2.1. 接口概述

初始化网络模块,在使用其他网络功能前必须先调用此函数。

2.2.2. 函数原型

success = network.init()

2.2.3. 参数描述

无。

2.2.4. 返回值描述

初始化成功返回 True。

2.2.5. 示例

```
import quectel

net = quectel.Network()
if net.init():
    print("网络初始化成功")
```

2.3. deinit

2.3.1. 接口概述

释放资源。

2.3.2. 函数原型

network.deinit()

2.3.3. 参数描述

无。

2.3.4. 返回值描述

无。

2.3.5. 示例

```
net.deinit()
```

2.4. query_usim

2.4.1. 接口概述

查询 USIM 卡（SIM 卡）是否正常。

2.4.2. 函数原型

```
ret = network.query_usim()
```

2.4.3. 参数描述

无。

2.4.4. 返回值描述

查询成功返回 True,否则返回 False。

2.4.5. 示例

```
if net.query_usim():
    print("SIM 卡正常")
else:
    print("SIM 卡异常")
```

2.5. attach

2.5.1. 接口概述

注册到蜂窝网络。

2.5.2. 函数原型

```
network.attach()
```

2.5.3. 参数描述

无。

2.5.4. 返回值描述

无。如果附着失败，会抛出 OSError 异常。

2.5.5. 示例

net.attach()

2.6. is_connected

2.6.1. 接口概述

检查当前网络连接状态。

2.6.2. 函数原型

is_connected = network.is_connected()

2.6.3. 参数描述

无。

2.6.4. 返回值描述

返回 True 表示已连接到网络，否则未连接到网络。

2.6.5. 示例

```
if net.is_connected():
    print("网络已连接")
else:
    print("网络未连接")
```

2.7. set_apn

2.7.1. 接口概述

设置网络接入点名称（APN）及相关认证信息。

2.7.2. 函数原型

network.set_apn(apn[, username, password])

2.7.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
apn	是	str	-	APN 名称

username	否	str	""	用户名
password	否	str	""	密码

2.7.4. 返回值描述

无。如果设置失败，会抛出 OSError 异常。

2.7.5. 示例

```
# 设置 APN，无用户名密码
net.set_apn("cmnet")

# 设置 APN，带用户名密码
net.set_apn("internet", "user", "pass")
```

2.8. set_scanmode

2.8.1. 接口概述

设置网络搜索模式（如仅 LTE、仅 GSM 等）。

2.8.2. 函数原型

network.set_scanmode(mode)

2.8.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mode	是	int	-	扫描模式，可使用预定义常量：Network.SCAN_AUTO，Network.SCAN_GSM_ONLY，Network.SCAN_WCDMA_ONLY，Network.SCAN_LTE_ONLY

2.8.4. 返回值描述

无。如果设置失败，会抛出 OSError 异常。

2.8.5. 示例

```
# 设置为仅 LTE 模式
net.set_scanmode(net.SCAN_LTE_ONLY)
```

2.9. set_ iotopmode

2.9.1. 接口概述

设置物联网操作模式（如 eMTC、NB-IoT 等）。

2.9.2. 函数原型

network.set_iotopmode(mode)

2.9.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mode	是	int	-	IoT 模式，可使用预定义常量：Network.IOTOP_EMMC (eMTC)，Network.IOTOP_NB (NB-IoT)，Network.IOTOP_EMMC_NB (eMTC 和 NB-IoT)

2.9.4. 返回值描述

无。如果设置失败，会抛出 OSError 异常。

2.9.5. 示例

```
# 设置为 NB-IoT 模式
net.set_iotopmode(net.IOTOP_NB)
```

3 GNSS 模块

3.1. GNSS

3.1.1. 接口概述

创建一个新的 GNSS 对象实例。

3.1.2. 函数原型

gnss = quectel.GNSS()

3.1.3. 参数描述

无。

3.1.4. 返回值描述

返回一个新的 GNSS 对象。

3.2. start

3.2.1. 接口概述

启动 GNSS 模块，开始定位。

3.2.2. 函数原型

success = gnss.start()

3.2.3. 参数描述

无。

3.2.4. 返回值描述

启动成功返回 True，失败返回 False。

3.2.5. 示例

```
import quectel

gnss = quectel.GNSS()
if gnss.start():
    print("GNSS 启动成功")
```

3.3. stop

3.3.1. 接口概述

停止 GNSS 模块。

3.3.2. 函数原型

gnss.stop()

3.3.3. 参数描述

无。

3.3.4. 返回值描述

无返回值。

3.4. get_location

3.4.1. 接口概述

获取当前地理位置信息。

3.4.2. 函数原型

location = gnss.get_location()

3.4.3. 参数描述

无。

3.4.4. 返回值描述

成功获取且有效定位时返回一个字典，包含以下键值对；否则返回 None。

字典键	数据类型	说明
utc_time	str	UTC 时间 (格式: hhmmss.ss)
latitude	float	纬度 (度)
longitude	float	经度 (度)
hdop	float	水平精度因子
altitude	float	海拔高度 (米)
fix_mode	str	定位模式: "2D" 或 "3D"
cog	float	对地航向 (度)
speed_kmh	float	对地速度 (公里/小时)
speed_knot	float	对地速度 (节)

字典键	数据类型	说明
date	str	日期 (格式: ddmmyy)
satellites	int	使用的卫星数量

3.4.5. 示例

```
import utime
import quectel

gnss = quectel.GNSS()
gnss.start()

# 等待定位
for i in range(10):
    utime.sleep(1)
    loc = gnss.get_location()
    if loc:
        print("定位成功!")
        print("纬度:", loc['latitude'])
        print("经度:", loc['longitude'])
        print("海拔:", loc['altitude'])
        print("模式:", loc['fix_mode'])
        print("时间:", loc['utc_time'], loc['date'])
        break
    else:
        print("正在定位...")

gnss.stop()
```

4 File 模块

4.1. File.open

4.1.1. 接口概述

创建一个文件对象对象实例，用于操作文件。

4.1.2. 函数原型

f = quectel.File.open(filename, mode='r')

4.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
filename	是	str	-	要打开的文件名
mode	否	str	'r'	文件打开模式: 'a' 若文件不存在，则会创建一个新文件；若文件存在，则直接打开文件。两种情况中的文件皆可读写。 'w' 若文件不存在，则会创建一个新文件；若文件存在，则清除并覆盖文件。两种情况中的文件皆可读写。 'r' 若文件存在，则打开文件，且文件只可读；若文件不存在，则响应错误。

4.1.4. 返回值描述

返回一个新的 File 对象。如果文件打开失败，会抛出 OSError 异常。

4.1.5. 示例

```
from quectel import File
f = File.open("test.txt", "r")
data = f.read(128)
print(data.decode('utf-8'))
f.close()
```

4.2. close

4.2.1. 接口概述

关闭文件对象。文件操作完成后应调用此方法以释放资源。

4.2.2. 函数原型

f.close()

4.2.3. 参数描述

无。

4.2.4. 返回值描述

无。

4.2.5. 示例

```
from quectel import File
f = File.open("test.txt", "w")
f.write(b"Hello")
f.close()
```

4.3. read

4.3.1. 接口概述

从文件中读取指定字节数的数据。

4.3.2. 函数原型

data = f.read(size)

4.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
size	是	int	-	要读取的最大字节数

4.3.4. 返回值描述

成功时返回一个 bytes 对象，包含读取到的数据。如果读取到文件末尾，则返回空 bytes 对象。失败时抛出 OSError 异常。

4.3.5. 示例

```
from quectel import File
f = File.open("test.txt", "a")
data = f.read(1024)
print(data.decode('utf-8'))
f.close()
```

4.4. write

4.4.1. 接口概述

向文件写入数据。

4.4.2. 函数原型

bytes_written = f.write(data)

4.4.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
data	是	bytes/bytearray	-	要读取的最大字节数

4.4.4. 返回值描述

成功时返回实际写入的字节数。失败时抛出 OSError 异常。

4.4.5. 示例

```
from quectel import File

# 3. 写入二进制数据（字节数组）
f = File.open("binary.dat", "w")

# 直接写字节值
f.write(b'\x00\x01\x02\x03\x04\x05')

# 写 ASCII 码
f.write(b'ABCDE')

# 写混合数据
data = bytes([0x48, 0x65, 0x6c, 0x6c, 0x6f])   # "Hello"的字节
f.write(data)

f.close()
```

4.5. seek

4.5.1. 接口概述

移动文件指针到新的位置。

4.5.2. 函数原型

new_position = file.seek(offset, whence)

4.5.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
offset	是	int	-	相对于 whence 的偏移量（字节）
whence	是	int	-	基准位置。 SEEK_SET:文件初始位置向后移动 SEEK_CUR:文件指针当前位置向后移动 SEEK_END:文件结尾位置向前移动

4.5.4. 返回值描述

返回移动后的文件指针位置（从文件开头计算的字节数）。失败时抛出 OSError 异常。

4.5.5. 示例

```
from quectel import File

f = File.open("binary.dat", "r")
length = f.seek(3, File.SEEK_SET)
data = f.read(1024)
f.close()
```

4.6. tell

4.6.1. 接口概述

获取当前文件指针的位置。

4.6.2. 函数原型

```
position = file.tell()
```

4.6.3. 参数描述

无。

4.6.4. 返回值描述

返回当前文件指针位置（从文件开头计算的字节数）。失败时抛出 OSError 异常。

4.6.5. 示例

```
from quectel import File
```

```
f = File.open("binary.dat", "r")
length = f.seek(0, File.SEEK_END)
pos = f.tell()
print("pos: ", pos)
f.close()
```

4.7. File.listdir

4.7.1. 接口概述

列出指定目录下的所有文件和子目录。

4.7.2. 函数原型

iterator = File.listdir(path)

4.7.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
path	是	str	-	要列出的目录路径(根目录为'*')

4.7.4. 返回值描述

返回一个迭代器，迭代器中的每个元素是一个 FileInfo 对象。

4.7.5. 示例

```
from quectel import File

# 列出根目录下的所有文件
for info in File.listdir('*'):
    print("文件名: {}, 大小: {} 字节".format(info.name(), info.size()))

# 也可以收集到列表中
files = list(File.listdir('dir2'))
print("共找到 {} 个文件/目录".format(len(files)))
```

4.8. 类 FileInfo

4.8.1. 接口概述

该类用于存储文件或目录的元信息，由 File.listdir() 返回的迭代器生成。

4.8.2. 函数原型

```
name = fileinfo.name()
size_bytes = fileinfo.size()
```

4.8.3. 参数描述

无。

4.8.4. 返回值描述

返回一个字符串，表示文件名或目录;
返回一个整数，表示文件大小,如果为-1 表示为目录。

4.8.5. 示例

```
from quectel import File

# 列出根目录下的所有文件或子目录
for info in File.listdir('*'):
    print("文件/目录名: {}, 大小: {} 字节".format(info.name(), info.size()))
```

4.9. File.statvfs

4.9.1. 接口概述

获取文件系统的空间使用情况。

4.9.2. 函数原型

```
total, free = quectel.File.statvfs()
```

4.9.3. 参数描述

无。

4.9.4. 返回值描述

返回一个包含两个元素的元组：(total_space, free_space)，单位均为字节。

4.9.5. 示例

```
from quectel import File

total, free = File.statvfs("UFS")
print("总空间: {} 字节".format(total))
print("剩余空间: {} 字节".format(free))
```

```
print("已使用: {} 字节".format(total - free))
print("使用率: {:.2f}%".format((total - free) / total * 100))
```

4.10. File.remove

4.10.1. 接口概述

删除指定的文件。

4.10.2. 函数原型

File.remove(filename)

4.10.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
filename	是	str	-	要删除的文件名

4.10.4. 返回值描述

无。如果文件不存在或删除失败，会抛出 OSError 异常。

4.10.5. 示例

```
from quectel import File

File.remove("test.txt")
```

4.11. File.mkdir

4.11.1. 接口概述

创建新目录。

4.11.2. 函数原型

quectel.File.mkdir(dir_path)

4.11.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
dir_path	是	str	-	要创建的目录路径

4.11.4. 返回值描述

无。如果创建失败，会抛出 TypeError 异常。

4.11.5. 示例

```
from quectel import File

File.mkdir('test')
```

4.12. File.rmdir

4.12.1. 接口概述

删除指定的目录。默认情况下只能删除空目录，可通过 force 参数强制删除非空目录。

4.12.2. 函数原型

quectel.File.rmdir(dir_path[, force=False])

4.12.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
dir_path	是	str	-	要删除的目录路径
force	否	bool	False	是否强制删除。 - False（默认）：只能删除空目录。 - True: 强制删除目录及其所有内容（包括子目录和文件）。

4.12.4. 返回值描述

无。如果删除失败，会抛出 TypeError 异常。

4.12.5. 示例

```
from quectel import File

f = File.open('test/test.txt', 'w')
f.close()
File.rmdir('test', True)
```

5 LOG 模块

5.1. Log

5.1.1. 接口概述

创建一个新的 Log 对象实例。

5.1.2. 函数原型

log = quectel.Log()

5.1.3. 参数描述

无。

5.1.4. 返回值描述

返回一个新的 Log 对象。

5.2. set_level

5.2.1. 接口概述

设置系统日志输出级别。
支持设置的日志级别如下：

- VERBOSE：最详细日志
- DEBUG：调试信息
- INFO：普通信息
- WARN：警告信息
- ERROR：错误信息
- NONE：关闭日志输出

5.2.2. 函数原型

result = log.set_level(level)

5.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
level	是	int	-	日志级别，取值范围为 0~5

5.2.4. 返回值描述

设置成功时返回 True，失败时返回 False。

5.2.5. 示例

```
from quectel import Log
log = Log()
log.set_level(Log.NONE) #关闭日志
```

5.3. get_level

5.3.1. 接口概述

获取当前系统日志输出级别。

5.3.2. 函数原型

level = log.get_level()

5.3.3. 参数描述

无。

5.3.4. 返回值描述

返回当前日志级别。